

INFORME SUDOKU

Elaborado por:

Jhonatan Mauricio Trujillo Parra ID: 948747

Presentado a:

Mg. Esteban Ernesto Morales Castro

NRC-10-81405

Corporación Universitaria Minuto de Dios

Ingeniería en sistemas

2026

MODELADO DEL PROBLEMA

Variables.

Cada celda vacía del tablero 9x9, identificada como (fila, columna) con fila, columna en $\{0, 1, 2, 3, 4, 5, 6, 7, 8\}$. Hasta 81 variables en total (las que están llenas son constantes).

Dominios.

Para cada variable (celda vacía), el dominio es $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Restricciones.

- Fila: Para cada fila, todas las celdas deben contener valores distintos.
- Columna: Para cada columna, todas las celdas deben contener valores distintos.
- Bloque: Para cada bloque 3x3, todas las celdas deben contener valores distintos.

FUNCIONES

`imprimir_tablero(tablero)`.

Esta función lo que hace es que recibe el tablero y lo imprime en la consola. Recorre fila por fila, y cuando llega a la fila 3 o 6 imprime una línea horizontal para separar los bloques. Para cada celda, si el valor es 0 imprime un punto, si no imprime el número.

Después de la columna 2 y 5 agrega el separador vertical | para delimitar los bloques horizontales.

```
def imprimir_tablero(tablero):
    for f in range(9):
        if f in (3, 6):
            print("-" * 21)
            fila = ""
        for c in range(9):
            if tablero[f][c] == 0:
                fila += ". "
            else:
                fila += str(tablero[f][c]) + " "
            if c in (2, 5):
                fila += "| "
        print(fila)
```

obtener_vecinos(fila, columna).

Esta función recibe la posición de una celda y devuelve todas las celdas que están relacionadas con ella, es decir, las que comparten su fila, su columna o su bloque 3×3. Lo hace en tres pasos: recorre toda la fila, luego toda la columna, luego el bloque 3×3 calculando dónde empieza con $(\text{fila} // 3) * 3$.

```
def obtener_vecinos(fila, columna):
    """Devuelve las posiciones de las celdas que comparten fila,
    columna o bloque con la celda dada."""
    vecinos = set()

    # Vecinos de la fila
    for c in range(9):
        if c != columna:
            vecinos.add((fila, c))

    # Vecinos de la columna
    for f in range(9):
        if f != fila:
            vecinos.add((f, columna))

    # Vecinos del bloque 3x3
    bloque_fila = (fila // 3) * 3 # fila donde empieza el bloque
    bloque_columna = (columna // 3) * 3 # columna donde empieza el bloque
    for f in range(bloque_fila, bloque_fila + 3):
        for c in range(bloque_columna, bloque_columna + 3):
            if (f, c) != (fila, columna):
                vecinos.add((f, c))

    return vecinos
```

valores_validos(tablero, fila, columna).

Recibe el tablero y la posición de una celda, y devuelve qué números del 1 al 9 puede tomar esa celda sin violar ninguna restricción. Llama a `obtener_vecinos` para saber qué celdas revisar, recorre cada una y si tiene un valor distinto de 0 lo agrega al conjunto usados. Al final retorna `{1, 2, 3, 4, 5, 6, 7, 8, 9}` - usados, es decir, los que quedan libres.

```
def valores_validos(tablero, fila, columna):
    usados = set() # Conjunto de valores usados

    for (f, c) in obtener_vecinos(fila, columna):
        if tablero[f][c] != 0:
            usados.add(tablero[f][c])

    return set(range(1, 10)) - usados # {1, 2, ..., 9} menos los usados
```

siguiente_vacia(tablero).

La función recorre el tablero en orden de lectura, es decir, fila 0 a 8, columna 0 a 8, y retorna la primera celda que tenga valor 0. Si no encuentra ninguna, retorna None, lo que da a significar que el tablero está completo.

```
def siguiente_vacia(tablero):
    for f in range(9):
        for c in range(9):
            if tablero[f][c] == 0:
                return (f, c)
    return None # no hay celdas vacías
```

siguiente_vacia_mrv(tablero).

Versión mejorada de siguiente_vacia. En lugar de retornar la primera celda vacía, recorre todas las celdas vacías, cuenta cuántos valores válidos tiene cada una, y retorna la que tenga menos opciones. Así el algoritmo empieza por las celdas más difíciles, detecta conflictos antes y evita explorar caminos que van a fallar.

```
def siguiente_vacia_mrv(tablero):
    valores_minimos = 9 # Más que el máximo posible
    mejor_celda = None

    for f in range(9):
        for c in range(9):
            if tablero[f][c] == 0:
                numero_valores = len(valores_validos(tablero, f, c))
                if numero_valores < valores_minimos:
                    valores_minimos = numero_valores
                    mejor_celda = (f, c)

    return mejor_celda
```

construir_dominios(tablero).

La función prepara el diccionario de dominios que necesita AC-3 para que se pueda imprimir en la consola. Recorre todas las celdas del tablero y para cada una crea una entrada en el diccionario dominios con clave (f, c). Si la celda ya tiene un valor, su dominio es ese valor solo {constante} o {k}. Si está vacía, su dominio son los valores válidos actuales calculados con valores_validos. Retorna ese diccionario completo.

```
def construir_dominios(tablero):
    dominios = {}
    for f in range(9):
        for c in range(9):
            if tablero[f][c] == 0:
                dominios[(f, c)] = valores_validos(tablero, f, c)
            else:
                dominios[(f, c)] = {tablero[f][c]} # Dominio fijo para celdas ya llenas
    return dominios
```

revisar(Xi, Xj, dominios).

Es el corazón de AC-3. Recibe dos celdas vecinas y el diccionario de dominios, y verifica si el arco ($X_i \rightarrow X_j$) es consistente. Recorre cada valor x del dominio de X_i y pregunta: ¿el dominio de X_j es exactamente $\{x\}$? Si sí, significa que X_j solo puede tomar ese valor, entonces X_i no puede tomarlo también — lo elimina del dominio de X_i . Retorna True si eliminó algo, False si no.

```
def revisar(Xi, Xj, dominios):
    """¿Cada valor de Xi tiene soporte en Xj?"""
    eliminado = False
    for x in set(dominios[Xi]): # Para cada valor posible de Xi
        if dominios[Xj] == {x}: # Si Xj solo tiene un valor posible y es x, entonces x no es válido para Xi
            dominios[Xi].discard(x) # Eliminar x del dominio de Xi
            eliminado = True
    return eliminado
```

ac3(dominios).

Implementa el algoritmo AC-3 completo. Primero mete todos los arcos posibles en una cola, es decir, cada par de celdas vecinas forma dos arcos, uno en cada dirección. Luego entra en un ciclo: saca un arco (X_i, X_j), llama a revisar. Si se eliminó algo del dominio de X_i , revisa si ese dominio quedó vacío, si sí, retorna False porque no hay solución. Si no quedó vacío, mete de vuelta en la cola los arcos de los vecinos de X_i para que también sean re-verificados. Cuando la cola se vacía, retorna True.

```
def ac3(dominios):
    # Meter todos los arcos en la cola
    cola = deque()
    for f in range(9):
        for c in range(9):
            for vecino in obtener_vecinos(f, c):
                cola.append(((f, c), vecino))

    while cola:
        Xi, Xj = cola.popleft() # sacar un arco

        if revisar(Xi, Xj, dominios): # si se eliminó algo de D(Xi)
            if len(dominios[Xi]) == 0:
                return False # dominio vacío → sin solución

            # Agregar arcos inversos para re-verificar
            for vecino in obtener_vecinos(*Xi):
                if vecino != Xj: # No agregar el arco que acabamos de revisar
                    cola.append((vecino, Xi))

    return True # todos los dominios son consistentes
```

aplicar_dominios(tablero, dominios).

Después de AC-3, los dominios reducidos están en el diccionario, pero el tablero todavía tiene los 0 originales. Esta función los sincroniza: recorre el diccionario y cada celda cuyo dominio quedó con exactamente un valor lo escribe en el tablero. Así el tablero queda actualizado con todo lo que AC-3 pudo resolver.

```
def aplicar_dominios(tablero, dominios):
    for (f, c), valores in dominios.items():
        if len(valores) == 1:
            tablero[f][c] = next(iter(valores)) # Asignar el único valor posible
```


BACKTRACKING SIN MRV VS BACKTRAKING CON MRV

Backtracking sin MRV.

El backtracking es una búsqueda en profundidad que asigna valores a las variables una a una. Si se detecta una violación de restricción, se retrocede al punto de elección anterior.

```
def backtracking(tablero, contador):
    contador[0] += 1
    vacia = siguiente_vacia(tablero)
    if not vacia:
        return True # Solución encontrada

    fila, columna = vacia
    for valor in valores_validos(tablero, fila, columna):
        tablero[fila][columna] = valor # Asignar valor

        if backtracking(tablero, contador):
            return True # Continuar con esta asignación

        tablero[fila][columna] = 0 # Desasignar (backtrack)

    return False # No se encontró solución con esta asignación
```

La función siguiente_vacia() selecciona celdas en orden de lectura: recorre el tablero de izquierda a derecha y de arriba hacia abajo, retornando la primera celda vacía que encuentra. No tiene en cuenta cuántas opciones tiene esa celda.

```
def siguiente_vacia(tablero):
    for f in range(9):
        for c in range(9):
            if tablero[f][c] == 0:
                return (f, c)
    return None # no hay celdas vacías
```

La función `valores_validos()` calcula el conjunto de valores permitidos para una celda recorriendo todos sus vecinos (fila, columna y bloque) y retornando {1 al 9} menos los valores ya usados:

```
def valores_validos(tablero, fila, columna):  
    usados = set() # Conjunto de valores usados  
  
    for (f, c) in obtener_vecinos(fila, columna):  
        if tablero[f][c] != 0:  
            usados.add(tablero[f][c])  
  
    return set(range(1, 10)) - usados # {1, 2, ..., 9} menos los usados
```

El resultado que nos da al ejecutar la solución del sudoku sin MRV es de 4,433 llamadas recursivas.

Backtracking con MRV.

En lugar de elegir la primera celda vacía disponible, selecciona la celda con el menor número de valores válidos posibles. El MRV trata de: entre más restringida es una variable, más urgente es asignarla primero. Al hacerlo, el algoritmo detecta los conflictos antes y evita explorar ramas del árbol de búsqueda que inevitablemente van a fallar; en este caso, se utiliza la función `siguiente_vacia_mrv(tablero)`.

```
def siguiente_vacia_mrv(tablero):
    valores_minimos = 9 # Más que el máximo posible
    mejor_celda = None

    for f in range(9):
        for c in range(9):
            if tablero[f][c] == 0:
                numero_valores = len(valores_validos(tablero, f, c))
                if numero_valores < valores_minimos:
                    valores_minimos = numero_valores
                    mejor_celda = (f, c)

    return mejor_celda
```

```
def backtracking_mrv(tablero, contador):
    contador[0] += 1
    vacia = siguiente_vacia_mrv(tablero)
    if not vacia:
        return True # Solución encontrada

    fila, columna = vacia
    for valor in valores_validos(tablero, fila, columna):
        tablero[fila][columna] = valor # Asignar valor

        if backtracking_mrv(tablero, contador):
            return True # Continuar con esta asignación

        tablero[fila][columna] = 0 # Desasignar (backtrack)

    return False # No se encontró solución con esta asignación

# La idea clave es: entre más restringida es una variable, más urgente es asignarla primero.
```

El resultado que nos da al ejecutar la solución del sudoku con MRV es de 52 llamadas recursivas.

A continuación, un cuadro comparativo con MRV, sin MRV y por AC-3

Estrategia	Llamadas recursivas	Observación
Backtracking sin MRV	4,433	Explora muchas ramas innecesarias
Backtracking con MRV	52	Elige la celda más restringida primero
AC-3 (preprocesamiento)	0	Resolvió el sudoku solo por propagación

Según el cuadro comparativo, MRV redujo las llamadas recursivas de 4,433 a 52, una reducción de aproximadamente de un 98.8% respecto al backtracking simple.

Esto se explica porque el sudoku tiene varias celdas con dominios muy pequeños desde el inicio. MRV las identifica y asigna primero, sin explorar caminos que van a terminar en conflicto. El backtracking sin MRV, en cambio, empieza por celdas con muchas opciones y descubre los conflictos tarde, forzando retrocesos y aumentando el tiempo de respuesta.

AC-3

AC-3 reduce los dominios de las variables antes de comenzar el backtracking. Para cada arco (X_i, X_j) , elimina de $D(X_i)$ los valores que no tienen soporte en $D(X_j)$. En el sudoku, la restricción entre dos celdas vecinas es siempre $X_i \neq X_j$.

Un valor x en $D(X_i)$ no tiene soporte en $D(X_j)$ cuando $D(X_j) = \{x\}$, es decir, X_j solo puede tomar ese valor. En ese caso, x se elimina de $D(X_i)$ porque ambas celdas no pueden valer lo mismo.

Si tras la eliminación el dominio de alguna celda queda vacío, AC-3 detecta que no hay solución posible sin necesitar backtracking.

Como resultado:

AC-3 resolvió las 51 celdas vacías del sudoku por propagación de restricciones, sin necesitar una sola llamada de backtracking.

CONCLUSION

El backtracking básico funciona correctamente pero explora demasiadas ramas del árbol de búsqueda.

MRV es una mejora simple con impacto enorme ya que redujo las llamadas en un 98.8% eligiendo siempre la celda más restringida.

El modelado del problema ayudo en el entendimiento del llenado del sudoku, lo cual dio detalles de sus variables, sus dominios y sus restricciones

AC-3 resultó ser el más eficiente, resolviendo las 51 celdas vacías sin necesitar backtracking. Para puzzles de mayor dificultad, AC-3 se puede combinar con backtracking para reducir el trabajo del algoritmo.